

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:100

Annexure A

EXPERIMENTS

Code of Conduct:

I N.SANDIYANITTIANANDAM(192571029) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

N.SANDIYA

Annexure A

EXPERIMENTS

Q1. You are developing a compiler-based desk calculator that evaluates arithmetic expressions using syntax-directed definitions (SDD).

The system should:

- (i) Accept an arithmetic expression with operators +, -, *, / and parentheses
- (ii) Use syntax-directed evaluation to compute values based on operator precedence
- (iii) Optimize evaluation by reducing unnecessary temporary computations
- (iv) Display the final result of the expression

Demonstrate how SDD rules (synthesized attributes) are used for expression evaluation.

SDD

Grammar with semantic rules:

$E \rightarrow E + T \{E.val = E1.val + T.val\}$

$E \rightarrow E - T \{E.val = E1.val - T.val\}$

$E \rightarrow T \{E.val = T.val\}$

$T \rightarrow T * F \{T.val = T1.val * F.val\}$

$T \rightarrow T / F \{T.val = T1.val / F.val\}$

$T \rightarrow F \{T.val = F.val\}$

$F \rightarrow (E) \{F.val = E.val\}$

$F \rightarrow \text{digit} \{F.val = \text{digit}\}$

ANSWER:

Aim:

To implement a C program for evaluating arithmetic expressions using Syntax-Directed Definition (SDD) with synthesized attributes and operator precedence.

Algorithm:

- Start the program.
- Read the arithmetic expression.
- Parse using E, T, and F.
- Evaluate * and / before + and -.
- Evaluate expressions inside parentheses.
- Use synthesized val attributes to calculate results.
- Display the final result.
- Stop.

Code:

```
#include <stdio.h>
#include <ctype.h>
char s[100];
int i = 0;
int E ();
int T ();
int F ();
int E ()
{
    int v = T ();
    while (s[i] == '+' || s[i] == '-')
    {
        char op = s[i++];
        int x = T();
        if (op == '+')
            v = v + x;
        else
            v = v - x;
    }
    return v;
}
int T ()
{
    int v = F ();
    while (s[i] == '*' || s[i] == '/')
    {
        char op = s[i++];
        int x = F ();
        if (op == '*')
            v = v * x;
        else
            v = v / x;
    }
    return v;
}
```

```

}
int F()
{
    int v;
    if (s[i] == '(')
    {
        i++;
        v = E ();
        i++;
    }
    else
    {
        v = s[i++] - '0';
    }
    return v;
}
int main ()
{
    printf("Enter expression: ");
    scanf("%s", s);
    printf("Result = %d\n", E ());
    return 0;
}

```

Output:

```

Enter arithmetic expression: 2+3*4
Final Result = 14.00

-----
Process exited after 26.28 seconds with return value 0
Press any key to continue . . .

```

Result:

Thus, the arithmetic expression was successfully evaluated using SDD and synthesized attributes with correct operator precedence.

Q2. In a compiler, postfix expressions are evaluated during code generation using stack-based bottom-up evaluation.

Write a C program that:

- (i) Evaluates a postfix expression
- (ii) Demonstrates S-attributed definition evaluation
- (iii) Shows how intermediate results are computed

ANSWER:**Aim:**

To write a C program to evaluate a postfix expression using stack-based bottom-up evaluation and demonstrate S-attributed definition with intermediate results.

Algorithm:

- Start the program.
- Read the postfix expression.
- Scan the expression from left to right.
- Push operands into the stack.
- For an operator, pop two operands and perform the operation.
- Display the intermediate result and push it back into the stack.
- The final stack value is the result.
- Stop.

Code:

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
int stack [100], top = -1;
```

```
void push (int x)
```

```
{
```

```
    stack[++top] = x;
```

```
}
```

```
int pop ()
```

```

{
    return stack[top--];
}

int main ()
{
    char exp [100];

    int i, a, b, r;

    printf("Enter postfix expression: ");

    scanf("%s", exp);

    for (i = 0; exp[i] != '\0'; i++)
    {
        if (isdigit(exp[i]))
        {
            push(exp[i] - '0');
        }
        else
        {
            b = pop();
            a = pop();
            switch (exp[i])
            {
                case '+': r = a + b; break;

                case '-': r = a - b; break;

                case '*': r = a * b; break;

                case '/': r = a / b; break;

            }

```

```

        printf("%d %c %d = %d\n", a, exp[i], b, r);

        push(r);

    }

}

printf("Final Result = %d\n", pop());

return 0;

}

```

Output:

```

Enter postfix expression: 23*54*+

Intermediate Results:
2 * 3 = 6
5 * 4 = 20
6 + 20 = 26

Final Result = 26

-----
Process exited after 30.83 seconds with return value 0
Press any key to continue . . . |

```

Result:

Thus, the postfix expression was successfully evaluated using stack-based bottom-up evaluation and S-attributed definition, with intermediate results displayed.

Q3. You are designing a parser that uses L-attributed definitions, where inherited attributes are passed from parent to child.

Write a C program that:

- i) Evaluates a simple expression like $a + b * c$
- (ii) Uses function parameters to simulate inherited attributes
- (iii). Ensures correct precedence handling
- (iv). Demonstrates top-down evaluation logic

Explain how inherited attributes influence computation.

ANSWER:**Aim:**

To write a C program to evaluate $a + b * c$ using L-attributed definitions, function parameters, and correct operator precedence.

Algorithm:

- Start the program.
- Read values of a, b, and c.
- Pass values through function parameters as inherited attributes.
- Evaluate $b * c$ first.
- Add the result to a.
- Display the final result.
- Stop.

Code:

```
#include <stdio.h>

/* Function parameter simulates inherited attribute */

int term (int b, int c)

{

    return b * c;

}

int expression (int a, int b, int c)

{

    int t;

    t = term (b, c); // b * c

    return a + t; // a + (b * c)

}

int main ()

{

    int a, b, c, result;

    printf("Enter values of a, b and c: ");
```



```

scanf("%d %d %d", &a, &b, &c);

result = expression (a, b, c);

printf("\nb * c = %d\n", b * c);

printf("a + (b * c) = %d\n", result);

printf("Final Result = %d\n", result);

return 0;

}

```

Output:

```

Enter values of a, b and c: 2 3 4
Expression: a + b * c
a = 2, b = 3, c = 4
Intermediate Result:
b * c = 12
a + (b * c) = 14
Final Result = 14
-----
Process exited after 17.4 seconds with return value 0
Press any key to continue . . . |

```

Result:

Thus, the expression $a + b * c$ was successfully evaluated using L-attributed definition, function parameters, and correct operator precedence.

Q4. In a compiler, it is important to check whether two type expressions are equivalent.

Write a C program that:

- (i) Accepts two type expressions (e.g., `int`, `float`, `int*`)
- (ii) Compares them for **type equivalence**
- (iii) Displays whether they are:
 - (a) Equivalent
 - (b) Not equivalent

Extend the logic to handle pointer types (`int*`, `float*`).

ANSWER:**Aim:**

To write a C program to compare two type expressions and check type equivalence, including pointer types.

Algorithm:

- Start the program.
- Read two type expressions.
- Check the base types (int, float).
- Check whether each type is a pointer.
- Compare both base type and pointer status.
- If both are same, display Equivalent.
- Otherwise, display Not Equivalent.
- Stop.

Code:

```
#include <stdio.h>

#include <string.h>

int main ()
{
    char t1[20], t2[20];

    int p1 = 0, p2 = 0;

    printf("Enter first type: ");

    scanf("%s", t1);

    printf("Enter second type: ");

    scanf("%s", t2);

    if (t1[strlen(t1) - 1] == '*')
    {
        p1 = 1;

        t1[strlen(t1) - 1] = '\0';
    }
}
```

```

    if (t2[strlen(t2) - 1] == '*')
    {
        p2 = 1;

        t2[strlen(t2) - 1] = '\0';
    }

    if (strcmp(t1, t2) == 0 && p1 == p2)

        printf("Equivalent\n");

    else

        printf("Not Equivalent\n");

    return 0;
}

```

Output:

```

Enter first type expression: int
Enter second type expression: int*

Type 1: int
Type 2: int*
Result: Not Equivalent

-----
Process exited after 28.17 seconds with return value 0
Press any key to continue . . . |

```

Result:

Thus, the program successfully checks type equivalence for basic and pointer types such as int, float, int*, and float*.

Q5. You are building a semantic analyzer that detects type errors in arithmetic expressions.

Write a C program that:

- (i). Accepts two operands and their types
- (ii). Checks whether the operation (e.g., +, *) is valid
- (iii). Reports:

(a). Valid expression

(b)Type error (e.g., `int + char*`)

Demonstrate how compilers detect semantic errors during type checking

ANSWER:

Aim:

To write a C program to perform type checking on arithmetic expressions and detect type errors.

Algorithm:

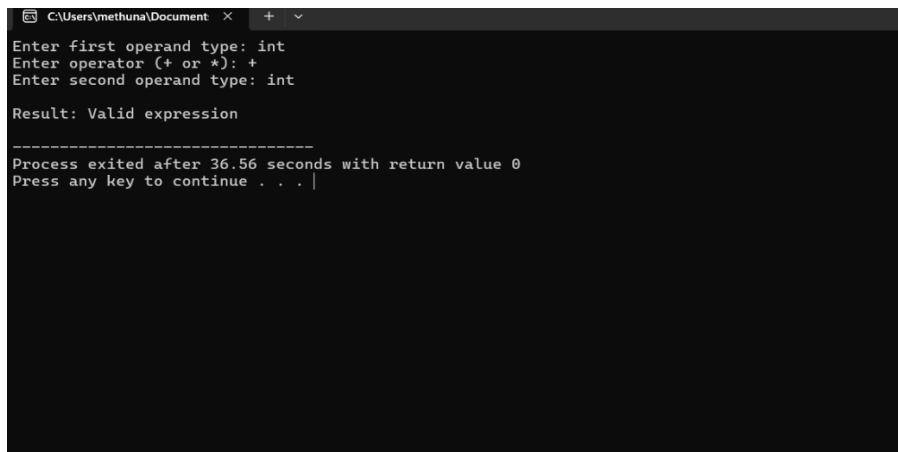
- Start the program.
- Read the first operand type.
- Read the operator (+ or *).
- Read the second operand type.
- Check whether both operands are compatible numeric types.
- If compatible, display Valid Expression.
- Otherwise, display Type Error.
- Stop.

Code:

```
#include <stdio.h>
#include <string.h>
int main ()
{
    char t1[20], t2[20], op;
    printf("Enter first operand type: ");
    scanf("%s", t1);
    printf("Enter operator (+ or *): ");
    scanf(" %c", &op);
    printf("Enter second operand type: ");
    scanf("%s", t2);
    if ((strcmp(t1, "int") == 0 || strcmp(t1, "float") == 0) &&
        (strcmp(t2, "int") == 0 || strcmp(t2, "float") == 0))
    {
        printf("Valid expression\n");
    }
}
```

```
else
{
    printf("Type error\n");
}
return 0;
}
```

Output:



```
C:\Users\methuna\Document >
Enter first operand type: int
Enter operator (+ or *): +
Enter second operand type: int
Result: Valid expression
-----
Process exited after 36.56 seconds with return value 0
Press any key to continue . . . |
```

Result:

Thus, the program successfully performs type checking and identifies valid expressions and semantic type errors such as `int + char*`.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation